1 Policies (read this first)

Deadline and late window. Lab2 is due by 11:59pm on Friday, October 23. Late submissions are accepted until 8:00am on Sunday, October 25 with an automatic 20% penalty. Nothing is accepted after that, for any reason. We always grade the last submission on Canvas, so a resubmission during the late window gets the penalty. Solutions are posted on Sunday, October 25.

AI policy. AI use (ChatGPT, Claude, Gemini, …) is allowed in this lab but must be disclosed: say in your README which tool you used and what for. Undisclosed AI use is an academic integrity violation. You must be able to explain, defend, and modify every line you submit, and the teaching team may quiz you on it.

Lab Check 2. In your lab section during week 5 (October 26 to 30), first 20 minutes: a short paper check on the Lab2 material, closed book, no devices, no AI. It is easy if you did and understood your own lab. No make-ups.

2 Goal

The goal of the second assignment is to create a PostgreSQL data schema with 7 tables that are very similar to the tables that you created in Lab1. The tables have the same names, attributes, and data types as the tables of Lab1, and the same Primary Keys and Foreign Keys, but they also have some UNIQUE constraints and restrictions on NULL that are described below.

After you create the data schema with the 7 tables, you must write five SQL statements that use those tables. Under Resources -> Lab2 on Piazza, we will provide you with data that you can load into your tables so that you can test the results of your queries. Testing can prove that a query is wrong, but not that it is right, so be careful. We will not give you the results of these queries on the load data; you should be able to figure out the results on that data yourselves. You can also test your queries on your own data. In the "real world", you have to make and check your own tests.

3 Lab2 Schema

3.1 Create PostgreSQL Schema Lab2

You will create a Lab2 schema to set apart the database tables created in this lab from the tables you will create in future, as well as from tables (and other objects) in the default (public) schema. In PostgreSQL, a database can have more than one schema; see the PostgreSQL documentation on schemas for more details. You create the Lab2 schema as follows:

```
CREATE SCHEMA Lab2;
```

Now that you have created the schema, you want to set Lab2 to be your default schema when you use psql. If you do not set Lab2 as the default schema, then you will have to qualify your table names with the schema name (e.g., Lab2.Payload). To set the default schema, you modify your search path.

```
ALTER ROLE cse180 SET SEARCH_PATH to Lab2;
```

You will need to log out and log back in to the server for this default schema change to take effect. (Type \q, then run the docker exec command from the Setup Guide again.) (Students often forget to do this.)

You do not have to include the CREATE SCHEMA or ALTER ROLE statements in your solution.

3.2 Create tables

As in Lab1, you will create the tables **Payload, LaunchSite, Mission, Astronaut, Contract, PayloadIntegration,** and **MissionCrew**. For reference, here they are again (the Primary Key of each

table is underlined):

Payload(payloadID, payloadName, owner, massKg, payloadType)

LaunchSite(siteCode, siteName, country, numLaunchPads, altitudeMeters)

Mission(missionID, launchSiteCode, scheduledLaunch, actualLaunch, rocketType, durationHours, outcome)

Astronaut(astronautID, astronautName, agency, dateOfBirth, yearsExperience)

Contract(contractID, missionID, signedDate, priority, price, paymentMethod)

PayloadIntegration(contractID, payloadID, integrationTime, numTechnicians, baySlot)

MissionCrew(missionID, astronautID, role, compensation)

The attributes for these tables are the same as for the tables of Lab1. Moreover, the data types for the attributes in these tables are the same as the ones specified for the tables of Lab1, and the Primary Keys and Foreign Keys are also the same. You may use the *create_lab1.sql* file solution that we provided on Piazza for Lab2 as the basis for the *create_lab2.sql* file which you include in your Lab2 solution (although you don't have to include our comments, and you're using Lab2 instead of Lab1 as the schema). However, your Lab2 tables must have additional constraints, which are described in the next section.

3.2.1 Constraints

The following attributes cannot be NULL. All other attributes can be NULL… but remember that attributes in Primary Keys also cannot be NULL, even though NOT NULL isn't specified for them.

- In Payload: payloadName
- In LaunchSite: siteName
- In Mission: scheduledLaunch, actualLaunch
- In Astronaut: astronautName

Also, the following must be unique for the specified table. That is, there cannot be identical rows in that table that have exactly the same (non-NULL) values for all of those attributes (composite unique constraint).

- In Payload: the 2 attributes payloadName and owner
- In LaunchSite: the 2 attributes siteName and country
- In Astronaut: the attribute astronautName
- In Contract: the 3 attributes signedDate, paymentMethod, and missionID

Explanations:

- The first constraint says that there can't be two rows in Payload which have the same values for both of the attributes payloadName and owner (if both of those attributes are not NULL). For example, there can't be two payloads whose payloadName is 'Starlink Batch 12', and whose owner is 'SpaceX'. But there could be two payloads whose payloadName is 'Starlink Batch 12' and whose owner is NULL.
- The second constraint says that there can't be two rows in LaunchSite which have the same values for siteName and country (recall that siteName here isn't allowed to be NULL).
- The third constraint says that there can't be two rows in Astronaut which have the same value for astronautName (recall that astronautName here isn't allowed to be NULL).
- The fourth constraint says that there can't be two rows in Contract which have the same values for all three of the attributes signedDate, paymentMethod, and missionID (if all three of those

attributes are not NULL). This means that we disallow duplicate "same-day, same mission, same payment method" contracts that appear to be the same deal.

Note on NULL in UNIQUE constraints: SQL treats NULL = NULL as UNKNOWN, not TRUE. So two Contract rows whose (signedDate, paymentMethod, missionID) values are (2026-01-12, NULL, 109) and (2026-01-12, NULL, 109) do not violate the fourth constraint; a UNIQUE constraint only rejects a row when all of its constrained values are non-NULL and identical to another row's.

You will write a CREATE TABLE command for each of the 7 tables which has these additional constraints. Save the commands in the file *create_lab2.sql*.

4 SQL Queries

Below are English descriptions of the five SQL queries that you need to write for this assignment, which you will include in files *queryX.sql*, where X is the number of the query, e.g., your SQL statement for Query 1 will be in the file *query1.sql*, and so forth. Follow the directions as given. You will lose points if you give extra tuples or attributes in your results, if you give attributes with the wrong names or in the wrong order, or if you have missing or wrong results. You will also lose points if your queries are unnecessarily complex, even if they are correct. Grading is based on correctness of queries on all data, not just the load data that we have provided.

Remember that the Referential Integrity (foreign key) constraints from Lab1 are still in effect here. For example, any contractID that is in a PayloadIntegration row must appear as a contractID in the Contract table.

Attributes should have their original names in the results of your queries, unless an alias is requested. And if a query asks that several attributes appear in the result, the first attribute mentioned should appear first, the second attribute mentioned should appear second, etc.

4.1 Query 1

Write a SQL query which finds the payloads whose name starts with 'S' (with that capitalization), which are integrated under a contract whose price is greater than 20000, where the mission of that contract launches from a launch site that has exactly 2 launch pads. The attributes in your result should be the name of the payload, the code of the launch site that the mission launches from, and the price of the contract, which should appear in your results as thePayloadName, theLaunchSiteCode, and thePrice.

No duplicates should appear in your result.

4.2 Query 2

A contract is funded by a government grant if its payment method (paymentMethod) is 'GovernmentGrant'.

Write a SQL query which finds all the missions that have at least one contract funded by a government grant, and that have never (in any contract, using any payment method) had a contract whose price is greater than 50000. The attributes in your result should be the mission ID and the rocket type, appearing as theMissionID and theRocketType. Tuples in your result should be in reverse alphabetical order by theRocketType; if two tuples have the same rocket type, they should appear in increasing order of theMissionID.

No duplicates should appear in your result.

4.3 Query 3

Recall that the assignment of an astronaut to a mission is recorded in MissionCrew.

Write a SQL query which finds all astronauts, excluding the astronaut whose name (astronautName) is 'Sophia Kim', who were assigned to the exact same rocket type on at least 2 different missions. The attribute in your result should be the astronaut ID, which should appear as theAstronautID.

No duplicates should appear in your result.

4.4 Query 4

Note: If myTimestamp is a TIMESTAMP, then in PostgreSQL, DATE(myTimestamp) is the DATE value which is in that timestamp. Unfortunately, different relational database systems have different ways of extracting the DATE from a TIMESTAMP. (There are at least 3 ways to do that in PostgreSQL; this one is the simplest.)

Recall that a tuple in PayloadIntegration identifies a payload being integrated into the rocket under a contract. Write a SQL query which finds all contracts for which payloads have been integrated and for which all of the following are true:

- The priority of the contract is high (priority = 'H').
- The date of integrationTime is May 1, 2026 or later.
- The name of the launch site that the contract's mission launches from has 'an' appearing anywhere in it (siteName), with that capitalization.
- The numTechnicians is greater than or equal to 5.
- The payment method used for the contract is NULL.

The attributes in your result should be the mission ID and the integrationTime, which should appear as theMissionID and theIntegrationTime.

No duplicates should appear in your result.

4.5 Query 5

Several crew assignments in the MissionCrew table might be linked to missions that share the same actual launch date. We want to find all the assignments which are associated with the earliest actual launch date. An assignment has the earliest date if there are no assignments whose associated mission has an earlier actual launch date.

Write a SQL query which finds the missionID and astronautID for all assignments whose associated mission has the earliest actual launch date. The attributes in your result should appear as theMissionID and theAstronautID.

No duplicates should appear in your result.

Note: Some missions are uncrewed: they have no tuples in MissionCrew. We want the earliest actual launch date among the missions that have at least one crew assignment, which is not necessarily the earliest actual launch date of all missions.

Note: As in Query 4, compare only the date portion of actualLaunch, not the full timestamp. Use `DATE(actualLaunch)`.

5 Testing

While your solution is still a work in progress, it is a good idea to drop all objects from the database every time you run the script, so you can start fresh. Of course, dropping each object may be tedious, and sometimes there may be a particular order in which objects must be dropped. The following commands (which you can put at the top of *create_lab2.sql* if you want, but you don't have to), will drop your Lab2 schema (and all objects within it), and then create the (empty) schema again:

```
DROP SCHEMA Lab2 CASCADE;

CREATE SCHEMA Lab2;
```

Before you submit, login to your database via psql and execute your script. As you've learned already, the command to execute a script is: `\i <filename>`. Remember that psql runs inside the Docker container, so keep your .sql files in your CSE180-lab folder (mounted at /CSE180-lab inside the container), and either run `\cd /CSE180-lab` once or give the full path, e.g., `\i /CSE180-lab/create_lab2.sql`.

Under Resources -> Lab2 on Piazza, we will post a load script named *load_lab2.sql* that loads data into the 7 tables of the database. You can execute that script with the command:

```
\i load_lab2.sql
```

You can test your 5 queries using that data, but you will have to figure out on your own whether your query results are correct. We won't provide answers to the 5 queries when they are run on the load data, and students should not share answers with other students. Also, your queries must be correct on any database instance, not just on the data that we provide. You may want to test your SQL statements on your own data as well.

6 DISTINCT Policy

In this course, DISTINCT must be used exactly when it is needed. All three of these cases lose points:

1. The query does NOT ask for duplicate removal, but you add DISTINCT anyway — this changes the required output and will be penalized.
2. The query asks for no duplicates, and your query could return duplicates without DISTINCT (e.g., the output attributes don't include a key of the rows that are joined) — omitting DISTINCT will produce incorrect results and will be penalized.
3. The query asks for no duplicates, but your query cannot return duplicates anyway (e.g., the output includes the Primary Key of the only table in FROM, or GROUP BY already produces one row per output value) — adding DISTINCT here is unnecessary and **will also be penalized**.

So “no duplicates should appear in your result” does not mean “always write DISTINCT”. For each query, ask yourself: can two different combinations of rows that satisfy my WHERE clause produce the same output row? If they can, you need DISTINCT (or GROUP BY). If they cannot, leave DISTINCT out. For example, `SELECT astronautID, astronautName FROM Astronaut WHERE agency = 'NASA'` cannot return duplicates, because astronautID is the Primary Key of Astronaut.

7 Submitting

1. Save your scripts for table creations and query statements as *create_lab2.sql* and *query1.sql* through *query5.sql*. You may add informative comments inside your scripts if you want (lines that start with two hyphens are interpreted as comment lines). Put any other information for the Graders in a separate README file that you may submit. If you used an AI tool, the README is where you disclose it (see Policies).

2. Zip the file(s) to a single file with name Lab2_XXXXXXX.zip where XXXXXXX is your 7-digit student ID. For example, if a student's ID is 1234567, then the file that this student submits for Lab2 should be named Lab2_1234567.zip

   To generate the zip file you can use the Unix command:

   ```
   zip Lab2_1234567 create_lab2.sql query1.sql query2.sql query3.sql
   query4.sql query5.sql README
   ```

   (Leave README out of the command if you don't have one. Of course, you use your own student ID, not 1234567.) Upload the zip file and, in the same Canvas submission, also upload each file individually (*create_lab2.sql*, *query1.sql* through *query5.sql*, and README if you have one). This lets us grade you even if the zip is broken. A submission with only a zip that turns out to be empty or wrong cannot be fixed after the solutions are posted.

3. Lab2 is due by 11:59pm on Friday, October 23. See Policies at the top of this document for the late window. Always check to make sure that your submission is on Canvas, and that you've submitted the correct files.
4. Be sure to follow directions about Academic Integrity that are in the Syllabus. If you have any questions about those directions, please speak to the instructor as soon as possible.